

Backpropagation and Empirical Gradient Computation for a Two-Policy Hybrid Contextual Bandit Using an ESN and Gaussian Policies

1 Problem setup and notation

We consider a contextual bandit problem. At each decision time t , the algorithm observes a context (or state) s_t , chooses an action $\mathbf{a}_t$, and immediately receives a reward r_t .

1.1 Basic symbols

- $t \in \{1, 2, \dots, B\}$: sample index inside a batch of size B .
- s_t : context (or state) at time t .
- $\mathbf{y}_t \in \mathbb{R}^{N_y}$: feature vector extracted from the context s_t . This is the input fed into the ESN.
- $\mathbf{z}_t \in \mathbb{R}^{N_z}$: ESN reservoir state at time t .
- $\mathbf{a}_t \in \mathbb{R}^{N_a}$: action taken at time t .
- $r_t \in \mathbb{R}$: immediate reward observed after taking action $\mathbf{a}_t$ in context s_t .

1.2 ESN parameters

- $\mathbf{W}_{\text{in}} \in \mathbb{R}^{N_z \times N_y}$: ESN input matrix.
- $\mathbf{W}_{\text{res}} \in \mathbb{R}^{N_z \times N_z}$: ESN reservoir matrix.
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{N_a \times N_z}$: ESN readout matrix.

Throughout this document we assume:

- $\mathbf{W}_{\text{in}}$ and $\mathbf{W}_{\text{res}}$ are *fixed*,
- only $\mathbf{W}_{\text{out}}$ is trainable.

Therefore, the parameter vector of the learned policy is

$$\phi \equiv \mathbf{W}_{\text{out}}.$$

1.3 Two-policy hybrid structure

We use two policies:

1. a learned policy, denoted by $\pi_\phi(\mathbf{a} \mid s)$,
2. a domain-knowledge policy, denoted by $\pi_{\text{DK}}(\mathbf{a} \mid s)$.

The overall policy is their mixture:

$$\pi_\theta(\mathbf{a} \mid s) = p \pi_\phi(\mathbf{a} \mid s) + (1 - p) \pi_{\text{DK}}(\mathbf{a} \mid s),$$

where

$$0 \leq p \leq 1$$

is the reuse probability of the learned policy.

1.4 Why we use α instead of p

Although the mixture can be written directly in terms of p , it is usually better to optimize an unconstrained scalar $\alpha \in \mathbb{R}$ and define

$$p = \text{sigmoid}(\alpha) = \frac{1}{1 + e^{-\alpha}}.$$

The reason is simple:

- the quantity p must always satisfy $0 \leq p \leq 1$,
- if we optimize p directly using ordinary gradient descent, an update step can easily push it outside the valid interval,
- in contrast, α is unconstrained, so it can take any real value,
- the sigmoid automatically maps every real α into a valid probability $p \in (0, 1)$.

So α is not a new policy quantity with a separate interpretation. It is simply a convenient *parameterization* of the probability p . Instead of optimizing a constrained variable p , we optimize an unconstrained variable α , and then recover

$$p = \text{sigmoid}(\alpha).$$

The full trainable parameter set is therefore

$$\theta = (\phi, \alpha) \quad \text{or equivalently} \quad \theta = (\mathbf{W}_{\text{out}}, \alpha).$$

2 Step 1: Forward pass

2.1 ESN state update equation

The ESN state update is

$$\mathbf{z}_t = f(\mathbf{W}_{\text{in}} \mathbf{y}_t + \mathbf{W}_{\text{res}} \mathbf{z}_{t-1}),$$

where $f(\cdot)$ denotes the *elementwise activation function* of the ESN reservoir, such as $\tanh(\cdot)$.

Here,

- $\mathbf{y}_t$ is the input feature vector at time t ,
- $\mathbf{z}_{t-1}$ is the previous reservoir state,
- $\mathbf{z}_t$ is the current reservoir state.

We assume an initial state $\mathbf{z}_0$ is given, for example $\mathbf{z}_0 = \mathbf{0}$.

Important observation. Since $\mathbf{W}_{\text{in}}$ and $\mathbf{W}_{\text{res}}$ are fixed, the reservoir state $\mathbf{z}_t$ is completely determined by the input sequence $\{\mathbf{y}_1, \dots, \mathbf{y}_t\}$. Therefore, when differentiating with respect to $\mathbf{W}_{\text{out}}$, the vector $\mathbf{z}_t$ behaves like a known feature vector.

2.2 ESN output equation

The ESN output is

$$\hat{\mathbf{x}}_t = \mathbf{W}_{\text{out}} \mathbf{z}_t,$$

where

$$\hat{\mathbf{x}}_t \in \mathbb{R}^{N_a}$$

is the mean action predicted by the learned policy at time t .

2.3 Learned Gaussian policy

We define the learned policy as a Gaussian distribution:

$$\pi_\phi(\mathbf{a}_t \mid s_t) = \mathcal{N}(\mathbf{a}_t; \hat{\mathbf{x}}_t, \sigma_a^2 \mathbf{I}_{N_a}),$$

where

- $\hat{\mathbf{x}}_t = \mathbf{W}_{\text{out}} \mathbf{z}_t$ is the mean,
- $\sigma_a^2 > 0$ is a fixed scalar action variance,
- $\mathbf{I}_{N_a}$ is the $N_a \times N_a$ identity matrix.

Thus the probability density is

$$\pi_\phi(\mathbf{a}_t \mid s_t) = \frac{1}{(2\pi\sigma_a^2)^{N_a/2}} \exp\left(-\frac{1}{2\sigma_a^2} \|\mathbf{a}_t - \mathbf{W}_{\text{out}} \mathbf{z}_t\|_2^2\right).$$

2.4 Why the domain-knowledge policy is written as a Gaussian

The domain-knowledge algorithm outputs a deterministic action

$$\mathbf{a}_t^{\text{DK}} = g_{\text{DK}}(s_t),$$

where $g_{\text{DK}}(\cdot)$ is a fixed domain-knowledge rule or optimization algorithm.

However, in the hybrid-policy formulation, we need $\pi_{\text{DK}}(\mathbf{a}_t \mid s_t)$ to be an ordinary probability density so that the mixture

$$\pi_\theta(\mathbf{a}_t \mid s_t) = p \pi_\phi(\mathbf{a}_t \mid s_t) + (1 - p) \pi_{\text{DK}}(\mathbf{a}_t \mid s_t)$$

is also a proper density and can be used inside $\log \pi_\theta(\mathbf{a}_t \mid s_t)$.

If we kept the DK policy as perfectly deterministic, then mathematically it would behave like a Dirac delta centered at the DK action. That is inconvenient for the likelihood-based mixture-policy gradient derivation used here. A common practical solution is therefore to approximate the deterministic DK policy by a *narrow Gaussian* centered at the DK action. This gives a smooth surrogate density that says, informally, “the DK branch strongly prefers actions near the DK action.”

So we define

$$\pi_{\text{DK}}(\mathbf{a}_t \mid s_t) = \mathcal{N}(\mathbf{a}_t; \boldsymbol{\mu}_t^{\text{DK}}, \sigma_{\text{DK}}^2 \mathbf{I}_{N_a}),$$

where

$$\boldsymbol{\mu}_t^{\text{DK}} = g_{\text{DK}}(s_t).$$

Thus the Gaussian assumption is not claiming that the DK algorithm itself is inherently Gaussian. It is only a convenient stochastic surrogate around the deterministic DK action.

2.5 Domain-knowledge Gaussian policy

Let the DK policy be represented as

$$\pi_{\text{DK}}(\mathbf{a}_t \mid s_t) = \mathcal{N}(\mathbf{a}_t; \boldsymbol{\mu}_t^{\text{DK}}, \sigma_{\text{DK}}^2 \mathbf{I}_{N_a}),$$

where

- $\boldsymbol{\mu}_t^{\text{DK}} \in \mathbb{R}^{N_a}$ is the DK action for context s_t ,
- $\sigma_{\text{DK}}^2 > 0$ is a small fixed variance.

This gives

$$\pi_{\text{DK}}(\mathbf{a}_t \mid s_t) = \frac{1}{(2\pi\sigma_{\text{DK}}^2)^{N_a/2}} \exp\left(-\frac{1}{2\sigma_{\text{DK}}^2} \|\mathbf{a}_t - \boldsymbol{\mu}_t^{\text{DK}}\|_2^2\right).$$

2.6 Mixture policy

The overall mixture policy is

$$\pi_{\theta}(\mathbf{a}_t \mid s_t) = p \pi_{\phi}(\mathbf{a}_t \mid s_t) + (1 - p) \pi_{\text{DK}}(\mathbf{a}_t \mid s_t),$$

with

$$p = \text{sigmoid}(\alpha) = \frac{1}{1 + e^{-\alpha}}.$$

This quantity is the probability density assigned by the hybrid policy to the observed action $\mathbf{a}_t$.

3 Step 2: Loss function

3.1 Bandit objective

In a contextual bandit, the objective is to maximize the expected immediate reward:

$$J(\theta) = \mathbb{E}_{(s, \mathbf{a}, r)}[r].$$

Because the action is sampled from the policy, the dependence of $J(\theta)$ on θ comes through the policy density.

3.2 Policy-gradient identity

For a stochastic policy, the standard score-function identity gives

$$\nabla_{\theta} J(\theta) = \mathbb{E}[r \nabla_{\theta} \log \pi_{\theta}(\mathbf{a} \mid s)].$$

Since you do not want reward centering, we use the raw reward directly, without subtracting any baseline.

3.3 Optimization as minimization of a loss

Neural-network optimization is usually written as minimization rather than maximization. Therefore, we define the loss as the negative objective:

$$\mathcal{L}(\theta) = -J(\theta).$$

At the batch level, the empirical loss is

$$\mathcal{L}_{\text{batch}}(\theta) = -\frac{1}{B} \sum_{t=1}^B r_t \log \pi_{\theta}(\mathbf{a}_t \mid s_t).$$

This is the quantity whose gradient we will compute.

Remark. The DK policy is fixed, but it still enters the loss through the mixture density

$$\pi_{\theta}(\mathbf{a}_t \mid s_t) = p \pi_{\phi}(\mathbf{a}_t \mid s_t) + (1 - p) \pi_{\text{DK}}(\mathbf{a}_t \mid s_t).$$

Therefore, even though π_{DK} is not trainable, it affects the gradients with respect to $\mathbf{W}_{\text{out}}$ and p .

4 Step 3: Gradient of the loss function

We now derive the gradients with respect to

- the ESN readout matrix $\mathbf{W}_{\text{out}}$,
- the mixture parameter p , and then
- the logit parameter α when $p = \text{sigmoid}(\alpha)$.

4.1 Log-density of the learned Gaussian policy

Start from the learned Gaussian density:

$$\pi_{\phi}(\mathbf{a}_t \mid s_t) = \frac{1}{(2\pi\sigma_a^2)^{N_a/2}} \exp\left(-\frac{1}{2\sigma_a^2} \|\mathbf{a}_t - \mathbf{W}_{\text{out}} \mathbf{z}_t\|_2^2\right).$$

Take the logarithm:

$$\log \pi_{\phi}(\mathbf{a}_t \mid s_t) = -\frac{N_a}{2} \log(2\pi\sigma_a^2) - \frac{1}{2\sigma_a^2} \|\mathbf{a}_t - \mathbf{W}_{\text{out}} \mathbf{z}_t\|_2^2.$$

The first term

$$-\frac{N_a}{2} \log(2\pi\sigma_a^2)$$

does not depend on $\mathbf{W}_{\text{out}}$, so its derivative is zero.

Therefore,

$$\nabla_{\mathbf{W}_{\text{out}}} \log \pi_{\phi}(\mathbf{a}_t \mid s_t) = \nabla_{\mathbf{W}_{\text{out}}} \left(-\frac{1}{2\sigma_a^2} \|\mathbf{a}_t - \mathbf{W}_{\text{out}} \mathbf{z}_t\|_2^2 \right).$$

4.2 Derivative of the quadratic term

Define the prediction error

$$\mathbf{e}_t = \mathbf{a}_t - \mathbf{W}_{\text{out}} \mathbf{z}_t.$$

Then

$$\|\mathbf{a}_t - \mathbf{W}_{\text{out}} \mathbf{z}_t\|_2^2 = \mathbf{e}_t^T \mathbf{e}_t.$$

Using the standard matrix derivative

$$\frac{\partial}{\partial \mathbf{W}} \|\mathbf{a} - \mathbf{W} \mathbf{z}\|_2^2 = -2(\mathbf{a} - \mathbf{W} \mathbf{z}) \mathbf{z}^T,$$

we obtain

$$\nabla_{\mathbf{W}_{\text{out}}} \log \pi_{\phi}(\mathbf{a}_t | s_t) = -\frac{1}{2\sigma_a^2} [-2(\mathbf{a}_t - \mathbf{W}_{\text{out}} \mathbf{z}_t) \mathbf{z}_t^T].$$

The factors $-\frac{1}{2\sigma_a^2}$ and -2 multiply to $\frac{1}{\sigma_a^2}$, so

$$\boxed{\nabla_{\mathbf{W}_{\text{out}}} \log \pi_{\phi}(\mathbf{a}_t | s_t) = \frac{1}{\sigma_a^2} (\mathbf{a}_t - \mathbf{W}_{\text{out}} \mathbf{z}_t) \mathbf{z}_t^T}$$

This is the key gradient for the ESN readout.

4.3 Why this is backpropagation

In this ESN setting, the learned policy mean is

$$\hat{\mathbf{x}}_t = \mathbf{W}_{\text{out}} \mathbf{z}_t.$$

Since $\mathbf{z}_t$ is fixed with respect to $\mathbf{W}_{\text{out}}$, the map

$$\mathbf{W}_{\text{out}} \mapsto \hat{\mathbf{x}}_t \mapsto \log \pi_{\phi}(\mathbf{a}_t | s_t)$$

is just a linear layer followed by the Gaussian log-density.

Therefore, *backpropagation* here is nothing more than repeated application of the chain rule through this simple computation graph. In fact, because only the readout is trainable, the result can be written in closed form exactly as above.

There is no backpropagation through time in this simplified ESN training setup because $\mathbf{W}_{\text{in}}$ and $\mathbf{W}_{\text{res}}$ are fixed.

4.4 Derivative of the mixture log-density with respect to $\mathbf{W}_{\text{out}}$

Recall that

$$\pi_{\theta}(\mathbf{a}_t | s_t) = p \pi_{\phi}(\mathbf{a}_t | s_t) + (1 - p) \pi_{\text{DK}}(\mathbf{a}_t | s_t).$$

We want

$$\nabla_{\mathbf{W}_{\text{out}}} \log \pi_{\theta}(\mathbf{a}_t | s_t).$$

By the chain rule for the logarithm,

$$\nabla_{\mathbf{W}_{\text{out}}} \log \pi_{\theta}(\mathbf{a}_t | s_t) = \frac{1}{\pi_{\theta}(\mathbf{a}_t | s_t)} \nabla_{\mathbf{W}_{\text{out}}} \pi_{\theta}(\mathbf{a}_t | s_t).$$

Since only π_ϕ depends on $\mathbf{W}_{\text{out}}$,

$$\nabla_{\mathbf{W}_{\text{out}}} \pi_\theta(\mathbf{a}_t \mid s_t) = p \nabla_{\mathbf{W}_{\text{out}}} \pi_\phi(\mathbf{a}_t \mid s_t).$$

Now use the identity

$$\nabla \pi = \pi \nabla \log \pi.$$

Therefore,

$$\nabla_{\mathbf{W}_{\text{out}}} \pi_\phi(\mathbf{a}_t \mid s_t) = \pi_\phi(\mathbf{a}_t \mid s_t) \nabla_{\mathbf{W}_{\text{out}}} \log \pi_\phi(\mathbf{a}_t \mid s_t).$$

Substituting into the previous expression gives

$$\nabla_{\mathbf{W}_{\text{out}}} \log \pi_\theta(\mathbf{a}_t \mid s_t) = \frac{p \pi_\phi(\mathbf{a}_t \mid s_t)}{\pi_\theta(\mathbf{a}_t \mid s_t)} \nabla_{\mathbf{W}_{\text{out}}} \log \pi_\phi(\mathbf{a}_t \mid s_t).$$

Define the *responsibility weight*

$$w_t = \frac{p \pi_\phi(\mathbf{a}_t \mid s_t)}{\pi_\theta(\mathbf{a}_t \mid s_t)}.$$

Then

$$\boxed{\nabla_{\mathbf{W}_{\text{out}}} \log \pi_\theta(\mathbf{a}_t \mid s_t) = w_t \nabla_{\mathbf{W}_{\text{out}}} \log \pi_\phi(\mathbf{a}_t \mid s_t)}$$

Substitute the ESN-Gaussian gradient derived earlier:

$$\boxed{\nabla_{\mathbf{W}_{\text{out}}} \log \pi_\theta(\mathbf{a}_t \mid s_t) = w_t \frac{1}{\sigma_a^2} (\mathbf{a}_t - \mathbf{W}_{\text{out}} \mathbf{z}_t) \mathbf{z}_t^T}$$

4.5 Gradient with respect to the reuse probability p

We now compute

$$\frac{\partial}{\partial p} \log \pi_\theta(\mathbf{a}_t \mid s_t).$$

Start from

$$\pi_\theta(\mathbf{a}_t \mid s_t) = p \pi_\phi(\mathbf{a}_t \mid s_t) + (1 - p) \pi_{\text{DK}}(\mathbf{a}_t \mid s_t).$$

Differentiate with respect to p :

$$\frac{\partial}{\partial p} \pi_\theta(\mathbf{a}_t \mid s_t) = \pi_\phi(\mathbf{a}_t \mid s_t) - \pi_{\text{DK}}(\mathbf{a}_t \mid s_t).$$

Then apply the derivative of the logarithm:

$$\boxed{\frac{\partial}{\partial p} \log \pi_\theta(\mathbf{a}_t \mid s_t) = \frac{\pi_\phi(\mathbf{a}_t \mid s_t) - \pi_{\text{DK}}(\mathbf{a}_t \mid s_t)}{\pi_\theta(\mathbf{a}_t \mid s_t)}}$$

4.6 Gradient with respect to the logit parameter α

Since

$$p = \text{sigmoid}(\alpha) = \frac{1}{1 + e^{-\alpha}},$$

we have

$$\frac{dp}{d\alpha} = p(1 - p).$$

By the chain rule,

$$\frac{\partial}{\partial \alpha} \log \pi_\theta(\mathbf{a}_t \mid s_t) = \frac{\partial}{\partial p} \log \pi_\theta(\mathbf{a}_t \mid s_t) \cdot \frac{dp}{d\alpha}.$$

Therefore,

$$\frac{\partial}{\partial \alpha} \log \pi_\theta(\mathbf{a}_t \mid s_t) = p(1-p) \frac{\pi_\phi(\mathbf{a}_t \mid s_t) - \pi_{\text{DK}}(\mathbf{a}_t \mid s_t)}{\pi_\theta(\mathbf{a}_t \mid s_t)}$$

4.7 Gradient of the batch loss

Recall that the batch loss is

$$\mathcal{L}_{\text{batch}}(\theta) = -\frac{1}{B} \sum_{t=1}^B r_t \log \pi_\theta(\mathbf{a}_t \mid s_t).$$

Differentiate with respect to $\mathbf{W}_{\text{out}}$:

$$\nabla_{\mathbf{W}_{\text{out}}} \mathcal{L}_{\text{batch}} = -\frac{1}{B} \sum_{t=1}^B r_t \nabla_{\mathbf{W}_{\text{out}}} \log \pi_\theta(\mathbf{a}_t \mid s_t).$$

Using the result above,

$$\nabla_{\mathbf{W}_{\text{out}}} \mathcal{L}_{\text{batch}} = -\frac{1}{B} \sum_{t=1}^B r_t w_t \frac{1}{\sigma_a^2} (\mathbf{a}_t - \mathbf{W}_{\text{out}} \mathbf{z}_t) \mathbf{z}_t^T$$

Similarly, differentiate with respect to α :

$$\nabla_\alpha \mathcal{L}_{\text{batch}} = -\frac{1}{B} \sum_{t=1}^B r_t \frac{\partial}{\partial \alpha} \log \pi_\theta(\mathbf{a}_t \mid s_t),$$

which gives

$$\nabla_\alpha \mathcal{L}_{\text{batch}} = -\frac{1}{B} \sum_{t=1}^B r_t p(1-p) \frac{\pi_\phi(\mathbf{a}_t \mid s_t) - \pi_{\text{DK}}(\mathbf{a}_t \mid s_t)}{\pi_\theta(\mathbf{a}_t \mid s_t)}$$

These are the gradients used for optimization.

5 Step 4: Empirical gradient computation from a batch

We now explain exactly how these gradients are computed from data.

5.1 Observed batch

Suppose we have a batch of B tuples:

$$\mathcal{B} = \{(s_t, \mathbf{a}_t, r_t)\}_{t=1}^B.$$

Each tuple contains:

- a context s_t ,
- an action $\mathbf{a}_t$,
- the immediate reward r_t .

5.2 Construct ESN features

For each sample t , compute the ESN state recursively:

$$\mathbf{z}_t = f(\mathbf{W}_{\text{in}}\mathbf{y}_t + \mathbf{W}_{\text{res}}\mathbf{z}_{t-1}).$$

Because the ESN reservoir is fixed, this is just a forward recursion.

5.3 Compute learned-policy mean

For each sample t , compute

$$\hat{\mathbf{x}}_t = \mathbf{W}_{\text{out}}\mathbf{z}_t.$$

This is the mean of the learned Gaussian policy.

5.4 Evaluate learned-policy density

For each sample t , evaluate

$$\pi_\phi(\mathbf{a}_t \mid s_t) = \frac{1}{(2\pi\sigma_a^2)^{N_a/2}} \exp\left(-\frac{1}{2\sigma_a^2}\|\mathbf{a}_t - \hat{\mathbf{x}}_t\|_2^2\right).$$

5.5 Evaluate DK density

For each sample t , evaluate

$$\pi_{\text{DK}}(\mathbf{a}_t \mid s_t) = \frac{1}{(2\pi\sigma_{\text{DK}}^2)^{N_a/2}} \exp\left(-\frac{1}{2\sigma_{\text{DK}}^2}\|\mathbf{a}_t - \boldsymbol{\mu}_t^{\text{DK}}\|_2^2\right).$$

5.6 Evaluate mixture density

For each sample t , compute

$$\pi_\theta(\mathbf{a}_t \mid s_t) = p\pi_\phi(\mathbf{a}_t \mid s_t) + (1-p)\pi_{\text{DK}}(\mathbf{a}_t \mid s_t).$$

5.7 Compute responsibility weights

For each sample t , define

$$w_t = \frac{p\pi_\phi(\mathbf{a}_t \mid s_t)}{\pi_\theta(\mathbf{a}_t \mid s_t)}.$$

This quantity measures how much of the mixture density at action $\mathbf{a}_t$ is explained by the learned policy.

5.8 Empirical gradient with respect to $\mathbf{W}_{\text{out}}$

Substitute everything into the batch gradient formula:

$$\nabla_{\mathbf{W}_{\text{out}}} \mathcal{L}_{\text{batch}} = -\frac{1}{B} \sum_{t=1}^B r_t w_t \frac{1}{\sigma_a^2} (\mathbf{a}_t - \mathbf{W}_{\text{out}}\mathbf{z}_t) \mathbf{z}_t^T$$

This is the empirical gradient of the loss with respect to the ESN readout matrix.

Interpretation. For each sample t :

- $(\mathbf{a}_t - \mathbf{W}_{\text{out}}\mathbf{z}_t)$ is the action error,
- $\mathbf{z}_t$ is the ESN feature vector,
- $(\mathbf{a}_t - \mathbf{W}_{\text{out}}\mathbf{z}_t)\mathbf{z}_t^T$ is an outer product,
- r_t scales the update directly according to the observed reward,
- w_t reduces the influence of samples that are better explained by the DK policy than by the learned policy.

Thus the update is a reward-weighted and mixture-weighted version of a linear regression gradient.

5.9 Empirical gradient with respect to α

Similarly,

$$\nabla_{\alpha}\mathcal{L}_{\text{batch}} = -\frac{1}{B}\sum_{t=1}^B r_t p(1-p) \frac{\pi_{\phi}(\mathbf{a}_t | s_t) - \pi_{\text{DK}}(\mathbf{a}_t | s_t)}{\pi_{\theta}(\mathbf{a}_t | s_t)}$$

This is the empirical gradient of the loss with respect to the reuse-logit parameter.

Interpretation. For each sample t :

- if $\pi_{\phi}(\mathbf{a}_t | s_t) > \pi_{\text{DK}}(\mathbf{a}_t | s_t)$, then the sample favors increasing the learned-policy reuse probability,
- if $\pi_{\phi}(\mathbf{a}_t | s_t) < \pi_{\text{DK}}(\mathbf{a}_t | s_t)$, then the sample favors decreasing it,
- the factor r_t determines how strongly that direction is weighted.

6 Parameter updates

After computing the gradients, perform gradient descent on the loss:

$$\mathbf{W}_{\text{out}} \leftarrow \mathbf{W}_{\text{out}} - \eta_W \nabla_{\mathbf{W}_{\text{out}}} \mathcal{L}_{\text{batch}},$$

$$\alpha \leftarrow \alpha - \eta_{\alpha} \nabla_{\alpha} \mathcal{L}_{\text{batch}},$$

where $\eta_W > 0$ and $\eta_{\alpha} > 0$ are learning rates.

Since $p = \text{sigmoid}(\alpha)$, the updated reuse probability is

$$p \leftarrow \text{sigmoid}(\alpha).$$

7 Summary of the full order of operations

For clarity, the full procedure for one batch is:

1. For each sample t , form the input feature $\mathbf{y}_t$.
2. Update the ESN state:

$$\mathbf{z}_t = f(\mathbf{W}_{\text{in}}\mathbf{y}_t + \mathbf{W}_{\text{res}}\mathbf{z}_{t-1}).$$

3. Compute the learned-policy mean:

$$\hat{\mathbf{x}}_t = \mathbf{W}_{\text{out}}\mathbf{z}_t.$$

4. Evaluate the learned-policy density:

$$\pi_\phi(\mathbf{a}_t \mid s_t) = \mathcal{N}(\mathbf{a}_t; \hat{\mathbf{x}}_t, \sigma_a^2 \mathbf{I}_{N_a}).$$

5. Evaluate the DK density:

$$\pi_{\text{DK}}(\mathbf{a}_t \mid s_t) = \mathcal{N}(\mathbf{a}_t; \boldsymbol{\mu}_t^{\text{DK}}, \sigma_{\text{DK}}^2 \mathbf{I}_{N_a}).$$

6. Form the mixture density:

$$\pi_\theta(\mathbf{a}_t \mid s_t) = p \pi_\phi(\mathbf{a}_t \mid s_t) + (1 - p) \pi_{\text{DK}}(\mathbf{a}_t \mid s_t).$$

7. Compute the responsibility weight:

$$w_t = \frac{p \pi_\phi(\mathbf{a}_t \mid s_t)}{\pi_\theta(\mathbf{a}_t \mid s_t)}.$$

8. Compute the batch loss:

$$\mathcal{L}_{\text{batch}} = -\frac{1}{B} \sum_{t=1}^B r_t \log \pi_\theta(\mathbf{a}_t \mid s_t).$$

9. Compute its gradients:

$$\nabla_{\mathbf{W}_{\text{out}}} \mathcal{L}_{\text{batch}} = -\frac{1}{B} \sum_{t=1}^B r_t w_t \frac{1}{\sigma_a^2} (\mathbf{a}_t - \mathbf{W}_{\text{out}}\mathbf{z}_t) \mathbf{z}_t^T,$$

$$\nabla_\alpha \mathcal{L}_{\text{batch}} = -\frac{1}{B} \sum_{t=1}^B r_t p(1 - p) \frac{\pi_\phi(\mathbf{a}_t \mid s_t) - \pi_{\text{DK}}(\mathbf{a}_t \mid s_t)}{\pi_\theta(\mathbf{a}_t \mid s_t)}.$$

10. Update $\mathbf{W}_{\text{out}}$ and α by gradient descent.

8 Final remarks

1. In this ESN setup, backpropagation is simple because only the readout $\mathbf{W}_{\text{out}}$ is trained. The reservoir states $\mathbf{z}_t$ are fixed features with respect to the trainable parameters.
2. The gradient with respect to $\mathbf{W}_{\text{out}}$ has the form of an outer product:

$$(\mathbf{a}_t - \mathbf{W}_{\text{out}}\mathbf{z}_t)\mathbf{z}_t^T,$$

which is exactly analogous to the gradient of a linear model.

3. The mixture-policy term w_t ensures that the learned policy is updated only in proportion to how much it contributes to the mixture density at the observed action.
4. The parameter α controls the reuse probability p , allowing the algorithm to automatically shift weight between the learned policy and the DK policy while always keeping p in the valid interval $(0, 1)$.
5. In this document, $f(\cdot)$ denotes the ESN activation function, while $\text{sigmoid}(\cdot)$ denotes the mapping from the unconstrained scalar α to the probability p . These are different functions with different purposes.